

React (software)

React (also known as **React.js** or **ReactJS**) is a free and open-source front-end JavaScript library^{[6][7]} that aims to make building user interfaces based on components more "seamless".^[6] It is maintained by Meta (formerly Facebook) and a community of individual developers and companies.^{[8][9][10]} According to the 2025 Stack Overflow Developer Survey, React is one of the most commonly used web technologies.^[11]

React can be used to develop single-page, mobile, or server-rendered applications with frameworks like Next.js and React Router. Because React is only concerned with the user interface and rendering components to the DOM, React applications often rely on libraries for routing and other client-side functionality.^{[12][13]} A key advantage of React is that it only re-renders those parts of the page that have changed, avoiding unnecessary re-rendering of unchanged DOM elements. React is used by an estimated 6% of all websites.^[14]

Features

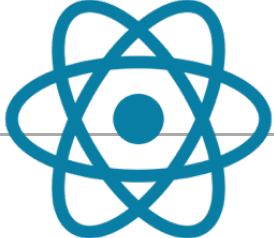
Declarative

React adheres to the declarative programming paradigm.^{[15][16]:76} Developers design views for each state of an application, and React updates and renders components when data changes. This is in contrast with imperative programming.^[17]

Components

React code is made of entities called components.^{[16]:10–12} These components are modular and can be reused.^{[16]:70} React applications typically consist of many layers of components. The components are rendered to a root element in the DOM using the React DOM library. When

React



Original author

Jordan Walke

Developers

Meta and community

Release

May 29, 2013^{[1][2]}

Stable release(s)

19.2.7^[3]  / 1 June 2026^[4]

Preview release(s)

19.0.0-rc.1 / November 14, 2024^[5]

Written in

JavaScript

Platform

Web platform

Type

JavaScript library

License

MIT License

Website

react.dev (https://react.dev/)

Repository

github.com/facebook/react (https://github.com/facebook/react)

rendering a component, values are passed between components through *props* (short for "properties"). Values internal to a component are called its *state*.^[18]

The two primary ways of declaring components in React are through function components and class components.^{[16]:118}^{[19]:10} Since React v16.8, using function components is the recommended way.

Function components

Function components, announced at React Conf 2018, and available since React v16.8, are declared with a function that accepts a single "props" argument and returns JSX (JavaScript XML). Function components can use internal state with the `useState` Hook.^[20]

React Hooks

On February 16, 2019, React 16.8 was released to the public, introducing React Hooks.^[20] Hooks are functions that let developers "hook into" React state and lifecycle features from function components.^[21] Notably, Hooks do not work inside classes — they let developers use more features of React without classes.^[22]

React provides several built-in hooks such as `useState`,^{[23][19]:37} `useContext`,^{[16]:11}^{[23][19]:12} `useReducer`,^{[16]:92}^{[23][19]:65–66} `useMemo`^{[16]:154}^{[23][19]:162} and `useEffect`.^{[24][19]:93–95} Others are documented in the Hooks API Reference.^{[25][16]:62} `useState` and `useEffect`, which are the most commonly used, are for controlling state^{[16]:37} and side effects,^{[16]:61} respectively.

Rules of hooks

There are two rules of hooks^[26] which describe the characteristic code patterns that hooks rely on:

1. "Only call hooks at the top level" — do not call hooks from inside loops, conditions, or nested statements so that the hooks are called in the same order each render.
2. "Only call hooks from React functions" — do not call hooks from plain JavaScript functions so that stateful logic stays with the component.

Although these rules cannot be enforced at runtime, code analysis tools such as linters can be configured to detect many mistakes during development. The rules apply to both usage of Hooks and the implementation of custom Hooks,^[27] which may call other Hooks.

Server components

React Server Components (RSC)^[28] are function components that run exclusively on the server. The concept was first introduced in the talk "Data Fetching with Server Components".^[29] Currently, server components are most readily usable with Next.js. With Next.js, it's

possible to write components for both the server and the client (browser). When a server rendered component is received by the browser, React in the browser takes over and creates the virtual DOM and attaches event handlers. This is called hydration.^[30]

Though a similar concept to Server Side Rendering, RSCs do not send corresponding JavaScript to the client as no hydration occurs. As a result, they have no access to hooks. However, they may be asynchronous function, allowing them to directly perform asynchronous operations:

```
1  async function MyComponent() {
2    const message = await fetchMessageFromDb();
3
4    return (
5      <div>Message: {message}</div>
6    );
7  }
```

Class components

Class components are declared using ES6 classes. They behave the same way that function components do, but instead of using Hooks to manage state and lifecycle events, they use the lifecycle methods on the `React.Component` base class.

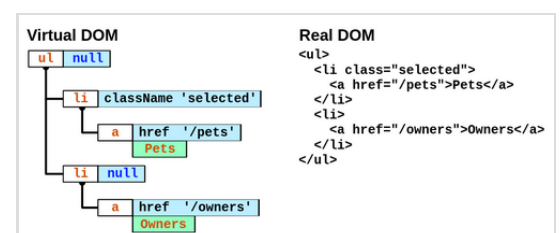
```
1  class ParentComponent extends React.Component {
2    state = { color: 'green' };
3    render() {
4      return (
5        <ChildComponent color={this.state.color} />
6      );
7    }
8  }
```

Routing

React itself does not come with built-in support for routing. React is primarily a library for building user interfaces, and it does not include a full-fledged routing solution out of the box. Third-party libraries can be used to handle routing in React applications, such as React Router.^[31] It allows the developer to define routes, manage navigation, and handle URL changes in a React-friendly way.

Virtual DOM

Another notable feature is the use of a virtual Document Object Model, or Virtual DOM. React creates an in-memory data-structure, similar to the browser DOM. Every time components are rendered, the result is compared with the virtual DOM. It then updates the browser's displayed DOM efficiently with only the computed differences.^[32] This process is called



There is a Virtual DOM that is used to implement the real DOM

reconciliation. This allows the programmer to write code as if the entire page is rendered on each change, while React only renders the components that actually change. This selective rendering provides a major performance boost.^{[33][34]}

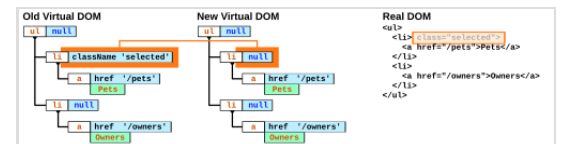
Updates

When `ReactDOM.render`^[35] is called again for the same component and target, React represents the new UI state in the Virtual DOM and determines which parts (if any) of the living DOM needs to change.^[36]

Lifecycle methods

Lifecycle methods for class-based components use a form of hooking that allows the execution of code at set points during a component's lifetime.

- `ShouldComponentUpdate` allows the developer to prevent unnecessary re-rendering of a component by returning false if a render is not required.
- `componentDidMount` is called once the component has "mounted" (the component has been created in the user interface, often by associating it with a DOM node). This is commonly used to trigger data loading from a remote source via an API.
- `componentDidUpdate` is invoked immediately after updating occurs.^[37]
- `componentWillUnmount` is called immediately before the component is torn down or "unmounted". This is commonly used to clear resource-demanding dependencies to the component that will not simply be removed with the unmounting of the component (e.g., removing any `setInterval()` instances that are related to the component, or an "event listener" set on the "document" because of the presence of the component)
- `render` is the most important lifecycle method and the only required one in any component. It is usually called every time the component's state is updated, which should be reflected in the user interface.



The virtualDOM will update the realDOM in real-time.

JSX

JSX, or JavaScript XML, is an extension to the JavaScript language syntax.^[38] Similar in appearance to HTML,^{[16]:11} JSX provides a way to structure component rendering using syntax familiar^{[16]:15} to many developers. React components are typically written using JSX, although they do not have to be (components may also be written in pure JavaScript). During compilation, JSX is converted to JavaScript code. JSX is similar to another extension syntax created by Facebook for PHP called XHP.

An example of JSX code:

```
function Example() {
  // Declare a new state variable, which we'll call "count"
  const [count, setCount] = useState(0);
```

```
return (  
  <div>  
    <p>You clicked {count} times</p>  
    <button onClick={() => setCount(count + 1)}>  
      Click me  
    </button>  
  </div>  
)  
};
```

Architecture beyond HTML

The basic architecture of React applies beyond rendering HTML in the browser. For example, Facebook has dynamic charts that render to `<canvas>` tags,^[39] and Netflix and PayPal use universal loading to render identical HTML on both the server and client.^{[40][41]} React can also be used to develop native apps for Android and iOS using React Native.

Server-side rendering

Server-side rendering (SSR) refers to the process of rendering a client-side JavaScript application on the server, rather than in the browser.^[42] This can improve the performance of the application, especially for users on slower connections or devices.^[43]

With SSR, the initial HTML that is sent to the client includes the fully rendered UI of the application.^[44] This allows the client's browser to display the UI immediately, rather than having to wait for the JavaScript to download and execute before rendering the UI.^[44]

React supports SSR, which allows developers to render React components on the server and send the resulting HTML to the client.^[45] This can be useful for improving the performance of the application, as well as for search engine optimization purposes.^[46]

Common idioms

React does not attempt to provide a complete application library. It is designed specifically for building user interfaces^[6] and therefore does not include many of the tools some developers might consider necessary to build an application. This allows the choice of whichever libraries the developer prefers to accomplish tasks such as performing network access or local data storage. Common patterns of usage have emerged as the library matures.

Unidirectional data flow

To support React's concept of unidirectional data flow (which might be contrasted with AngularJS's bidirectional flow), the *Flux* architecture was developed as an alternative to the popular model–view–controller architecture. Flux features *actions* which are sent through a

central *dispatcher* to a *store*, and changes to the store are propagated back to the view.^[47] When used with React, this propagation is accomplished through component properties. Since its conception, Flux has been superseded by libraries such as Redux and MobX.^[48]

Flux can be considered a variant of the observer pattern.^[49]

A React component under the Flux architecture should not directly modify any props passed to it, but should be passed callback functions that create *actions* which are sent by the dispatcher to modify the store. The action is an object whose responsibility is to describe what has taken place: for example, an action describing one user "following" another might contain a user id, a target user id, and the type `USER_FOLLOWED_ANOTHER_USER`.^[50] The stores, which can be thought of as models, can alter themselves in response to actions received from the dispatcher.

This pattern is sometimes expressed as "properties flow down, actions flow up". Many implementations of Flux have been created since its inception, perhaps the most well-known being Redux, which features a single store, often called a single source of truth.^[51]

In February 2019, `useReducer` was introduced as a React hook in the 16.8 release. It provides an API that is consistent with Redux, enabling developers to create Redux-like stores that are local to component states.^[52]

History

React was created by Jordan Walke, a software engineer at Facebook, Inc. (now Meta), who initially developed a prototype called "F-Bolt"^[53] before later renaming it to "FaxJS". This early version is documented in Jordan Walke's GitHub repository. Influences for the project included XHP, an HTML component library for PHP.

React was first deployed on Facebook's News Feed in 2011 and subsequently integrated into Instagram in 2012.^[54] In May 2013, at JSConf US, the project was officially open-sourced, marking a significant turning point in its adoption and growth.

React Native, which enables native Android, iOS, and UWP development with React, was announced at Facebook's React Conf in February 2015 and open-sourced in March 2015.

On April 18, 2017, Facebook announced React Fiber, a new set of internal algorithms for rendering, as opposed to React's old rendering algorithm, Stack.^[55] React Fiber was to become the foundation of any future improvements and feature development of the React library.^[56] The actual syntax for programming with React does not change; only the way that the syntax is executed has changed.^[57] React's old rendering system, Stack, was developed at a time when the focus of the system on dynamic change was not understood. Stack was slow to draw complex animation, for example, trying to accomplish all of it in one chunk. Fiber breaks down animation into segments that can be spread out over multiple frames. Likewise, the structure of

a page can be broken into segments that may be maintained and updated separately. JavaScript functions and virtual DOM objects are called "fibers", and each can be operated and updated separately, allowing for smoother on-screen rendering.^[58]

On September 26, 2017, React 16.0 was released to the public.^[59] React 16.0 introduced error boundaries, a new component type that catches JavaScript errors anywhere in its child tree and renders a fallback UI instead of crashing the app.^[60]

On October 20, 2020, the React team released React v17.0, notable as the first major release without major changes to the React developer-facing API.^[61]

On March 29, 2022, React 18 was released which introduced a new concurrent renderer, automatic batching and support for server side rendering with Suspense.^[62] React 18 dropped support for Internet Explorer 11.^[63]

On December 5, 2024, React 19 was released. This release introduced Actions, which simplify the process of making state updates using asynchronous functions rather than having to manually handle pending states, errors and optimistic updates. React 19 also included support for server components and improved static site generation.^[64]

In October 2025, Meta announced that it would donate React, React Native, and JSX (JavaScript XML) to a new React Foundation, part of the Linux Foundation.^[65] On 24 February 2026, The React Foundation officially launched with Meta Platforms, Inc. contributing the React project to the Foundation.^[66]

On 29 November 2025, a vulnerability CVE-2025-55182, also known as React2Shell was reported that allowed remote code execution. It was assigned a CVSS highest score of 10.0.^[67]^[68] A fix was introduced in versions 19.0.1, 19.1.2, and 19.2.1.^[69]

On December 11, 2025, the React team disclosed additional vulnerabilities in React Server Components: denial-of-service issues (CVE-2025-55184 and CVE-2025-67779, CVSS 7.5) and a source code exposure issue (CVE-2025-55183, CVSS 5.3).^[70]^[71]^[72]^[73]^[74] Fixes were backported to versions 19.0.3, 19.1.4, and 19.2.3.^[75]

Version history of React

Licensing

The initial public release of React in May 2013 used the [Apache License 2.0](#). In October 2014, React 0.12.00 replaced this with the [3-clause BSD license](#) and added a separate PATENTS text file that permits usage of any Facebook patents related to the software.^[77]

The license granted hereunder will terminate, automatically and without notice, for anyone that makes any claim (including by filing any lawsuit, assertion or other action) alleging (a) direct, indirect, or contributory infringement or inducement to infringe any patent: (i) by Facebook or any of its subsidiaries or affiliates, whether or not such claim is related to the Software, (ii) by any party if such claim arises in whole or in part from any software, product or service of Facebook or any of its subsidiaries or affiliates, whether or not such claim is related to the Software, or (iii) by any party relating to the Software; or (b) that any right in any patent claim of Facebook is invalid or unenforceable.

This unconventional clause caused some controversy and debate in the React user community, because it could be interpreted to empower Facebook to revoke the license in many scenarios, for example, if Facebook sues the licensee prompting them to take "other action" by publishing the action on a blog or elsewhere. Many expressed concerns that Facebook could unfairly exploit the termination clause or that integrating React into a product might complicate a startup company's future acquisition.^[78]

Based on community feedback, Facebook updated the patent grant in April 2015 to be less ambiguous and more permissive:^[79]

The license granted hereunder will terminate, automatically and without notice, if you (or any of your subsidiaries, corporate affiliates or agents) initiate directly or indirectly, or take a direct financial interest in, any Patent Assertion: (i) against Facebook or any of its subsidiaries or corporate affiliates, (ii) against any party if such Patent Assertion arises in whole or in part from any software, technology, product or service of Facebook or any of its subsidiaries or corporate affiliates, or (iii) against any party relating to the Software. [...] A "Patent Assertion" is any lawsuit or other action alleging direct, indirect, or contributory infringement or inducement to infringe any patent, including a cross-claim or counterclaim.^[80]

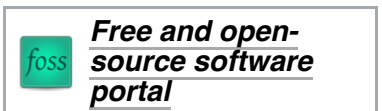
The [Apache Software Foundation](#) considered this licensing arrangement to be incompatible with its licensing policies, as it "passes along risk to downstream consumers of our software imbalanced in favor of the licensor, not the licensee, thereby violating our Apache legal policy of being a universal donor", and "are not a subset of those found in the [Apache License 2.0], and they cannot be sublicensed as [Apache License 2.0]".^[81] In August 2017, Facebook dismissed the Apache Foundation's downstream concerns and refused to reconsider their license.^{[82][83]} The following month, [WordPress](#) decided to switch its Gutenberg and Calypso projects away from React.^[84]

On September 23, 2017, Facebook announced that the following week, it would re-license Flow, Jest, React, and Immutable.js under a standard MIT License; the company stated that React was "the foundation of a broad ecosystem of open source software for the web", and that they did not want to "hold back forward progress for nontechnical reasons".^[85]

On September 26, 2017, React 16.0.0 was released with the MIT license.^[86] The MIT license change has also been backported to the 15.x release line with React 15.6.2.^[87]

See also

- Angular (web framework)
- Backbone.js
- Ember.js
- Gatsby (JavaScript framework)
- Next.js
- TypeScript
- Svelte
- Vue.js
- Comparison of JavaScript-based web frameworks
- Web Components



Notes

References

- Occhino, Tom; Walke, Jordan (5 August 2013). "JS Apps at Facebook" (<https://www.youtube.com/watch?v=GW0rj4sNH2w>). *YouTube*. Archived (<https://web.archive.org/web/20220531133559/https://www.youtube.com/watch?v=GW0rj4sNH2w>) from the original on 31 May 2022. Retrieved 22 Oct 2018.
- "Is React a Library or a Framework? Here's Why it Matters" (<https://www.freecodecamp.org/news/is-react-a-library-or-a-framework/>). *freeCodeCamp.org*. 2021-04-12. Retrieved 2024-10-12.
- "Release 19.2.7" (<https://github.com/facebook/react/releases/tag/v19.2.7>). 1 June 2026. Retrieved 2 June 2026.
- <https://github.com/facebook/react/tags>
- "What's new in React 19" (<https://react.dev/blog/2024/04/25/react-19#whats-new-in-react-19>). Archived (<https://web.archive.org/web/20240512195006/https://react.dev/blog/2024/04/25/react-19#whats-new-in-react-19>) from the original on 2024-05-12. Retrieved 2024-05-12.
- "React – A JavaScript library for building user interfaces" (<https://reactjs.org>). *reactjs.org*. Archived (<https://web.archive.org/web/20180408084010/https://reactjs.org/>) from the original on April 8, 2018. Retrieved 7 April 2018.
- Chapter 1. What Is React? - What React Is and Why It Matters [Book]* (<https://www.oreilly.co>

m/library/view/what-react-is/9781491996744/ch01.html). ISBN 978-1-4919-9674-4. Archived (<https://web.archive.org/web/20230506100446/https://www.oreilly.com/library/view/what-react-is/9781491996744/ch01.html>) from the original on May 6, 2023. Retrieved 2023-05-06.

{{cite book}}: |website= ignored (help)

8. Krill, Paul (May 15, 2014). "React: Making faster, smoother UIs for data-driven Web apps" (<https://www.infoworld.com/article/2608181/javascript/react--making-faster--smoother-uis-for-data-driven-web-apps.html>). *InfoWorld*. Archived (<https://web.archive.org/web/20180612141516/https://www.infoworld.com/article/2608181/javascript/react--making-faster--smoother-uis-for-data-driven-web-apps.html>) from the original on 2018-06-12. Retrieved 2021-02-23.
9. Hemel, Zef (June 3, 2013). "Facebook's React JavaScript User Interfaces Library Receives Mixed Reviews" (<https://www.infoq.com/news/2013/06/facebook-react>). *infoq.com*. Archived (<https://web.archive.org/web/20220526082114/https://www.infoq.com/news/2013/06/facebook-react/>) from the original on May 26, 2022. Retrieved 2022-01-11.
10. Dawson, Chris (July 25, 2014). "JavaScript's History and How it Led To ReactJS" (<https://thenewstack.io/javascripts-history-and-how-it-led-to-reactjs/>). *The New Stack*. Archived (<https://web.archive.org/web/20200806190027/https://thenewstack.io/javascripts-history-and-how-it-led-to-reactjs/>) from the original on Aug 6, 2020. Retrieved 2020-07-19.
11. "2025 Stack Overflow Developer Survey" (<https://survey.stackoverflow.co/2025/technology/#1-web-frameworks-and-technologies>). *Stack Overflow*. Archived (<https://web.archive.org/web/20251216042301/https://survey.stackoverflow.co/2025/technology/#1-web-frameworks-and-technologies>) from the original on 2025-12-16. Retrieved 2025-10-10.
12. Dere 2017.
13. Panchal 2022.
14. "Usage statistics and market share of React for websites" (<https://w3techs.com/technologies/details/js-react>). *w3techs.com*. October 7, 2025. Retrieved October 7, 2025.
15. "React Introduction" (<https://www.geeksforgeeks.org/reactjs-introduction/>). *GeeksforGeeks*. 2017-09-27. Retrieved 2024-10-12.
16. Wieruch 2020.
17. Schwarzmüller 2018.
18. "Components and Props" (<https://reactjs.org/docs/components-and-props.html#props-are-read-only>). *React*. Facebook. Archived (<https://web.archive.org/web/20180407120115/https://reactjs.org/docs/components-and-props.html>) from the original on 7 April 2018. Retrieved 7 April 2018.
19. Larsen 2021.
20. "Introducing Hooks" (<https://reactjs.org/docs/hooks-intro.html>). *react.js*. Archived (<https://web.archive.org/web/20181025163202/https://reactjs.org/docs/hooks-intro.html>) from the original on 2018-10-25. Retrieved 2019-05-20.
21. "Hooks at a Glance – React" (<https://reactjs.org/docs/hooks-overview.html>). *reactjs.org*. Archived (<https://web.archive.org/web/20230315054047/https://reactjs.org/docs/hooks-overview.html>) from the original on 2023-03-15. Retrieved 2019-08-08.
22. "What the Heck is React Hooks?" (<https://soshace.com/2020/01/16/what-the-heck-is-react-hooks/>). *Soshace*. 2020-01-16. Archived (<https://web.archive.org/web/20220531133601/https://blog.soshace.com/what-the-heck-is-react-hooks/>) from the original on 2022-05-31. Retrieved 2020-01-24.
23. "Using the State Hook – React" (<https://reactjs.org/docs/hooks-state.html>). *reactjs.org*. Archived (<https://web.archive.org/web/20220730180312/https://reactjs.org/docs/hooks-state.html>) from the original on 2022-07-30. Retrieved 2020-01-24.

24. "Using the Effect Hook – React" (<https://reactjs.org/docs/hooks-effect.html>). *reactjs.org*. Archived (<https://web.archive.org/web/20220801212858/https://reactjs.org/docs/hooks-effect.html>) from the original on 2022-08-01. Retrieved 2020-01-24.
25. "Hooks API Reference – React" (<https://reactjs.org/docs/hooks-reference.html>). *reactjs.org*. Archived (<https://web.archive.org/web/20220805061010/https://reactjs.org/docs/hooks-reference.html>) from the original on 2022-08-05. Retrieved 2020-01-24.
26. "Rules of Hooks – React" (<https://reactjs.org/docs/hooks-rules.html>). *reactjs.org*. Archived (<https://web.archive.org/web/20210606174151/https://reactjs.org/docs/hooks-rules.html>) from the original on 2021-06-06. Retrieved 2020-01-24.
27. "Building Your Own Hooks – React" (<https://reactjs.org/docs/hooks-custom.html>). *reactjs.org*. Archived (<https://web.archive.org/web/20220717175155/https://reactjs.org/docs/hooks-custom.html>) from the original on 2022-07-17. Retrieved 2020-01-24.
28. "React Labs: What We've Been Working On – March 2023" (<https://react.dev/blog/2023/03/22/react-labs-what-we-have-been-working-on-march-2023#react-server-components>). *react.dev*. Archived (<https://web.archive.org/web/20230726201006/https://react.dev/blog/2023/03/22/react-labs-what-we-have-been-working-on-march-2023#react-server-components>) from the original on 2023-07-26. Retrieved 2023-07-23.
29. Abramov, Dan; Tan, Lauren; Savona, Joseph; Markbåge, Sebastian (2020-12-21). "Introducing Zero-Bundle-Size React Server Components" (<https://react.dev/blog/2020/12/21/data-fetching-with-react-server-components>). *react.dev*. Retrieved 2024-09-28.
30. "hydrate" (<https://18.react.dev/reference/react-dom/hydrate#hydrating-server-rendered-html>). Archived (<https://web.archive.org/web/20240716002720/https://18.react.dev/reference/react-dom/hydrate#hydrating-server-rendered-html>) from the original on 2024-07-16. Retrieved 2025-06-19.
31. "Mastering React Router – The Ultimate Guide" (<https://www.devban.com/react-router-ultimate-guide/>). 2023-07-12. Archived (<https://web.archive.org/web/20230726063450/https://www.devban.com/react-router-ultimate-guide/>) from the original on 2023-07-26. Retrieved 2023-07-26.
32. "Refs and the DOM" (<https://reactjs.org/docs/refs-and-the-dom.html>). *React Blog*. Archived (<https://web.archive.org/web/20220807171328/https://reactjs.org/docs/refs-and-the-dom.html>) from the original on 2022-08-07. Retrieved 2021-07-19.
33. "React: The Virtual DOM" (<https://www.codecademy.com/articles/react-virtual-dom>). *Codecademy*. Archived (<https://web.archive.org/web/20211028172953/https://www.codecademy.com/articles/react-virtual-dom>) from the original on 2021-10-28. Retrieved 2021-10-14.
34. Aggarwal, Sanchit (March 2018). "Modern Web-Development using ReactJS" (<https://ijrra.net/Vol5issue1/IJRRRA-05-01-27.pdf>) (PDF). *International Journal of Recent Research Aspects*. pp. 133–137. Archived (<https://web.archive.org/web/20240417143754/https://ijrra.net/Vol5issue1/IJRRRA-05-01-27.pdf>) (PDF) from the original on 17 April 2024. Retrieved 11 December 2024.
35. "ReactDOM – React" (<https://reactjs.org/docs/react-dom.html>). *reactjs.org*. Archived (<https://web.archive.org/web/20230108104936/https://reactjs.org/docs/react-dom.html>) from the original on 2023-01-08. Retrieved 2023-01-08.
36. "Reconciliation – React" (<https://reactjs.org/docs/reconciliation.html>). *reactjs.org*. Archived (<https://web.archive.org/web/20230108105122/https://reactjs.org/docs/reconciliation.html>) from the original on 2023-01-08. Retrieved 2023-01-08.
37. "React.Component – React" (<https://legacy.reactjs.org/docs/react-component.html>). *legacy.reactjs.org*. Archived (<https://web.archive.org/web/20240409075058/https://legacy.reactjs.org/docs/react-component.html>) from the original on 2024-04-09. Retrieved

2024-04-09.

38. "Draft: JSX Specification" (<https://facebook.github.io/jsx/>). *JSX*. Facebook. 2022-03-08. Archived (<https://web.archive.org/web/20220402072504/https://facebook.github.io/jsx/>) from the original on 2022-04-02. Retrieved 7 April 2018.
39. Hunt, Pete (2013-06-05). "Why did we build React? – React Blog" (<https://web.archive.org/web/20150406072833/http://facebook.github.io/react/blog/2013/06/05/why-react.html>). *reactjs.org*. Archived from the original (<https://facebook.github.io/react/blog/2013/06/05/why-react.html>) on 2015-04-06. Retrieved 2022-02-17.
40. "PayPal Isomorphic React" (<https://medium.com/paypal-engineering/isomorphic-react-apps-with-react-engine-17dae662379c>). *medium.com*. 2015-04-27. Archived (<https://web.archive.org/web/20190208124143/https://www.paypal-engineering.com/2015/04/27/isomorphic-react-apps-with-react-engine/>) from the original on 2019-02-08. Retrieved 2019-02-08.
41. "Netflix Isomorphic React" (<http://techblog.netflix.com/2015/01/netflix-likes-react.html>). *netflixtechblog.com*. 2015-01-28. Archived (<https://web.archive.org/web/20161217043150/http://techblog.netflix.com/2015/01/netflix-likes-react.html>) from the original on 2016-12-17. Retrieved 2022-02-14.
42. "Server-side rendering (SSR) - MDN Web Docs Glossary" (<https://developer.mozilla.org/en-US/docs/Glossary/SSR>). *MDN Web Docs*. Mozilla. Retrieved 7 March 2025.
43. "Rendering on the Web" (<https://web.dev/rendering-on-the-web/>). *web.dev*. Google. 6 February 2019. Retrieved 7 March 2025.
44. Jain, Atishay (10 November 2018). "Render Caching for React" (<https://css-tricks.com/render-caching-for-react/>). *CSS-Tricks*. Retrieved 7 March 2025.
45. "Server React DOM APIs" (<https://react.dev/reference/react-dom/server>). *React Documentation*. Meta Platforms. Retrieved 7 March 2025.
46. "Rendering (Next.js Documentation)" (<https://nextjs.org/docs/pages/building-your-application/rendering>). *Next.js Documentation*. Vercel. Retrieved 7 March 2025.
47. "In Depth OverView" (<https://web.archive.org/web/20220807201252/https://facebook.github.io/flux/docs/in-depth-overview/>). *Flux*. Facebook. Archived from the original (<https://facebook.github.io/flux/docs/in-depth-overview/>) on 7 August 2022. Retrieved 7 April 2018.
48. "Flux Release 4.0" (<https://github.com/facebook/flux/releases/tag/4.0.0>). *Github*. Archived (<https://web.archive.org/web/20220531133558/https://github.com/facebook/flux/releases/tag/4.0.0>) from the original on 31 May 2022. Retrieved 26 February 2021.
49. Johnson, Nicholas. "Introduction to Flux – React Exercise" (<http://nicholasjohnson.com/react/course/exercises/flux/>). *Nicholas Johnson*. Archived (<https://web.archive.org/web/20220531133600/http://nicholasjohnson.com/react/course/exercises/flux/>) from the original on 31 May 2022. Retrieved 7 April 2018.
50. Abramov, Dan. "The History of React and Flux with Dan Abramov" (<http://threedevsandamaybe.com/the-history-of-react-and-flux-with-dan-abramov/>). *Three Devs and a Maybe*. Archived (<https://web.archive.org/web/20180419075905/http://threedevsandamaybe.com/the-history-of-react-and-flux-with-dan-abramov/>) from the original on 19 April 2018. Retrieved 7 April 2018.
51. "State Management Tools – Results" (<http://2016.stateofjs.com/2016/statemanagement/>). *The State of JavaScript*. Archived (<https://web.archive.org/web/20220531133609/http://2016.stateofjs.com/2016/statemanagement/>) from the original on 31 May 2022. Retrieved 29 October 2021.
52. "React v16.8: The One with Hooks" (<https://reactjs.org/blog/2019/02/06/react-v16.8.0.html#react-1>). Archived (<https://web.archive.org/web/20230108090021/https://reactjs.org/blog/2019/02/06/react-v16.8.0.html#react-1>) from the original on 2023-01-08. Retrieved 2023-01-08.

53. "React.js: The Documentary" (https://youtube.com/watch?v=8pDqJVdNa44%3Fsi%3DFMJqegC4dPtWKP__&t=528). *Youtube*. Honeypot. 10 February 2023. Archived (https://web.archive.org/web/20240119211307/https://www.youtube.com/watch?v=8pDqJVdNa44%3Fsi%3DFMJqegC4dPtWKP__&t=528) from the original on 2024-01-19. Retrieved 2024-05-27.
54. Lopez, Marny (13 May 2024). "Why React is so widely adopted by web developers?" (<https://www.devlane.com/blog/why-react-is-so-widely-adopted-by-web-developers>). *Devlane*. Archived (<https://web.archive.org/web/20240620092857/https://www.devlane.com/blog/why-react-is-so-widely-adopted-by-web-developers>) from the original on 20 June 2024. Retrieved 11 December 2024.
55. Lardinois 2017.
56. "React Fiber Architecture" (<https://github.com/acdlite/react-fiber-architecture>). *Github*. Archived (<https://web.archive.org/web/20180510140634/https://github.com/acdlite/react-fiber-architecture>) from the original on 10 May 2018. Retrieved 19 April 2017.
57. "Facebook announces React Fiber, a rewrite of its React framework" (<https://techcrunch.com/2017/04/18/facebook-announces-react-fiber-a-rewrite-of-its-react-framework/>). *TechCrunch*. 18 April 2017. Archived (<https://web.archive.org/web/20180614172053/https://techcrunch.com/2017/04/18/facebook-announces-react-fiber-a-rewrite-of-its-react-framework/>) from the original on 2018-06-14. Retrieved 2018-10-19.
58. "GitHub – acdlite/react-fiber-architecture: A description of React's new core algorithm, React Fiber" (<https://github.com/acdlite/react-fiber-architecture>). *github.com*. Archived (<https://web.archive.org/web/20180510140634/https://github.com/acdlite/react-fiber-architecture>) from the original on 2018-05-10. Retrieved 2018-10-19.
59. "React v16.0" (<https://reactjs.org/blog/2017/09/26/react-v16.0.html>). *react.js*. 2017-09-26. Archived (<https://web.archive.org/web/20171003031315/https://reactjs.org/blog/2017/09/26/react-v16.0.html>) from the original on 2017-10-03. Retrieved 2019-05-20.
60. "React v16.0 – React Blog" (<https://legacy.reactjs.org/blog/2017/09/26/react-v16.0.html>). *legacy.reactjs.org*. Retrieved 2025-10-15.
61. "React v17.0 Release Candidate: No New Features – React Blog" (<https://legacy.reactjs.org/blog/2020/08/10/react-v17-rc.html>). *legacy.reactjs.org*. Retrieved 2026-05-25.
62. "React 18" (<https://react.dev/blog/2022/03/29/react-v18>). *React*. Retrieved 7 December 2024.
63. "How to Upgrade to React 18 – React" (<https://react.dev/blog/2022/03/08/react-18-upgrade-guide>). *react.dev*. Retrieved 2025-11-14.
64. "React 19" (<https://react.dev/blog/2024/12/05/react-19#whats-new-in-react-19>). *React*. Retrieved 7 December 2024.
65. "Meta will move React to Linux Foundation to address vendor dominance fears" (https://www.theregister.com/2025/10/09/meta_react_foundation/). *The Register*. 9 October 2025.
66. "Linux Foundation Announces the Formation of the React Foundation" (<https://www.linuxfoundation.org/press/linux-foundation-announces-the-formation-of-the-react-foundation>). *Linux Foundation*. 2026-02-24. Retrieved 2026-02-25.
67. "CVE-2025-55182 Detail on National Vulnerability Database website" (<https://nvd.nist.gov/vuln/detail/CVE-2025-55182>). Retrieved 2025-12-05.
68. Toulas, Bill. "Critical React, Next.js flaw lets hackers execute code on servers" (<https://www.bleepingcomputer.com/news/security/critical-react2shell-flaw-in-react-nextjs-lets-hackers-run-javascript-code/>). *BleepingComputer*. Retrieved 2025-12-05.
69. "Critical Security Vulnerability in React Server Components on react.dev website" (<https://react.dev/blog/2025/12/03/critical-security-vulnerability-in-react-server-components>).

Retrieved 2025-12-05.

70. "Denial of Service and Source Code Exposure in React Server Components" (<https://react.dev/blog/2025/12/11/denial-of-service-and-source-code-exposure-in-react-server-components>). *react.dev*. Retrieved 2025-12-15.
71. "CVE-2025-55184 Detail - NVD" (<https://nvd.nist.gov/vuln/detail/CVE-2025-55184>). *National Vulnerability Database*. NIST. Retrieved 2025-12-15.
72. "CVE-2025-55183 Detail - NVD" (<https://nvd.nist.gov/vuln/detail/CVE-2025-55183>). *National Vulnerability Database*. NIST. Retrieved 2025-12-15.
73. "CVE-2025-67779 Detail - NVD" (<https://nvd.nist.gov/vuln/detail/CVE-2025-67779>). *National Vulnerability Database*. NIST. Retrieved 2025-12-15.
74. Lyons, Jessica (2025-12-12). "New React vulns leak secrets, invite DoS attacks" (https://www.theregister.com/2025/12/12/new_react_secretleak_bugs/). *The Register*. Retrieved 2025-12-15.
75. "Denial of Service and Source Code Exposure in React Server Components" (<https://react.dev/blog/2025/12/11/denial-of-service-and-source-code-exposure-in-react-server-components>). *react.dev*. Retrieved 2025-12-15.
76. "React v18.0" (<https://reactjs.org/blog/2022/03/29/react-v18.html>). *reactjs.org*. Archived (<https://web.archive.org/web/20220329160934/https://reactjs.org/blog/2022/03/29/react-v18.html>) from the original on 2022-03-29. Retrieved 2022-04-12.
77. "React CHANGELOG.md" (<https://github.com/facebook/react/blob/master/CHANGELOG.md#0120-october-28-2014>). *GitHub*. Archived (<https://web.archive.org/web/20200428042800/https://github.com/facebook/react/blob/master/CHANGELOG.md#0120-october-28-2014>) from the original on 2020-04-28. Retrieved 2015-12-09.
78. Liu, Austin. "A compelling reason not to use ReactJS" (<https://medium.com/bits-and-pixels/a-compelling-reason-not-to-use-reactjs-beac24402f7b>). *Medium*. Archived (<https://web.archive.org/web/20220531133320/https://medium.com/bits-and-pixels/a-compelling-reason-not-to-use-reactjs-beac24402f7b>) from the original on 2022-05-31. Retrieved 2015-12-09.
79. "Updating Our Open Source Patent Grant" (<https://code.facebook.com/posts/1639473982937255/updates-our-open-source-patent-grant/>). Archived (<https://web.archive.org/web/20201108114832/https://code.facebook.com/posts/1639473982937255/updates-our-open-source-patent-grant/>) from the original on 2020-11-08. Retrieved 2015-12-09.
80. "Additional Grant of Patent Rights Version 2" (<https://github.com/facebook/react/blob/b8ba8c83f318b84e42933f6928f231dc0918f864/PATENTS>). *GitHub*. Archived (<https://web.archive.org/web/20220531133320/https://github.com/facebook/react/blob/b8ba8c83f318b84e42933f6928f231dc0918f864/PATENTS>) from the original on 2022-05-31. Retrieved 2015-12-09.
81. "ASF Legal Previously Asked Questions" (<https://www.apache.org/legal/resolved.html>). Apache Software Foundation. Archived (<https://web.archive.org/web/20180206232339/http://www.apache.org/legal/resolved.html>) from the original on 2018-02-06. Retrieved 2017-07-16.
82. "Explaining React's License" (<https://code.facebook.com/posts/112130496157735/explaining-react-s-license/>). *Facebook*. Archived (<https://web.archive.org/web/20210506164245/https://code.facebook.com/posts/112130496157735/explaining-react-s-license/>) from the original on 2021-05-06. Retrieved 2017-08-18.
83. "Consider re-licensing to AL v2.0, as RocksDB has just done" (<https://github.com/facebook/react/issues/10191#issuecomment-323486580>). *Github*. Archived (<https://web.archive.org/web/20220727100939/https://github.com/facebook/react/issues/10191#issuecomment-323486580>) from the original on 2022-07-27. Retrieved 2017-08-18.
84. "WordPress to ditch React library over Facebook patent clause risk" (<https://techcrunch.com/2022/07/27/wordpress-to-ditch-react-library-over-facebook-patent-clause-risk/>). Retrieved 2022-07-27.

84. "WordPress to ditch React library over Facebook patent clause risk" (<https://techcrunch.com/2017/09/15/wordpress-to-ditch-react-library-over-facebook-patent-clause-risk/>). *TechCrunch*. 15 September 2017. Archived (<https://web.archive.org/web/20220531133350/https://techcrunch.com/2017/09/15/wordpress-to-ditch-react-library-over-facebook-patent-clause-risk/>) from the original on 2022-05-31. Retrieved 2017-09-16.
85. "Relicensing React, Jest, Flow, and Immutable.js" (<https://code.facebook.com/posts/300798627056246/relicensing-react-jest-flow-and-immutable-js/>). *Facebook Code*. 2017-09-23. Archived (<https://web.archive.org/web/20201206105641/https://code.facebook.com/posts/300798627056246/relicensing-react-jest-flow-and-immutable-js/>) from the original on 2020-12-06. Retrieved 2017-09-23.
86. Clark, Andrew (September 26, 2017). "React v16.0\$MIT licensed" (<https://reactjs.org/blog/2017/09/26/react-v16.0.html#mit-licensed>). *React Blog*. Archived (<https://web.archive.org/web/20171003031315/https://reactjs.org/blog/2017/09/26/react-v16.0.html#mit-licensed>) from the original on October 3, 2017. Retrieved October 18, 2017.
87. Hunzaker, Nathan (September 25, 2017). "React v15.6.2" (<https://reactjs.org/blog/2017/09/25/react-v15.6.2.html>). *React Blog*. Archived (<https://web.archive.org/web/20220531133328/https://reactjs.org/blog/2017/09/25/react-v15.6.2.html>) from the original on May 31, 2022. Retrieved October 18, 2017.

Bibliography

- Larsen, John (2021). *React Hooks in Action With Suspense and Concurrent Mode*. Manning. ISBN 978-1-72004-399-7.
- Schwarzmüller, Max (2018-05-01). *React – The Complete Guide (incl. Hooks, React Router and Redux)*. Packt Publishing.
- Wieruch, Robin (2020). *The Road to React*. Leanpub. ISBN 978-1-72004-399-7.
- Dere, Mohan (2017-12-21). "How to integrate create-react-app with all the libraries you need to make a great app" (<https://www.freecodecamp.org/news/how-to-make-create-react-app-work-with-a-node-backend-api-7c5c48acb1b0/>). *freeCodeCamp*. Retrieved 2018-06-14.
- Panchal, Krupal (2022-04-26). "Angular vs React Detailed Comparison" (<https://www.sitepoint.com/angular-vs-react/>). *SitePoint*. Archived (<https://web.archive.org/web/20230330160838/https://www.sitepoint.com/angular-vs-react/>) from the original on 2023-03-30. Retrieved 2023-06-05.
- Hámori, Fenerec (2022-05-31). "The History of React.js on a Timeline" (<https://blog.risingstack.com/the-history-of-react-js-on-a-timeline/>). *RisingStack*. Archived (<https://web.archive.org/web/20220531133616/https://blog.risingstack.com/the-history-of-react-js-on-a-timeline/>) from the original on 2022-05-31. Retrieved 2023-06-05.
- Lardinois, Frederic (2017-04-18). "Facebook announces React Fiber, a rewrite of its React framework" (<https://techcrunch.com/2017/04/18/facebook-announces-react-fiber-a-rewrite-of-its-react-framework/>). *TechCrunch*. Retrieved 2024-12-31.

External links

- Official website (<https://react.dev/>) 
 - react (<https://github.com/facebook/react>) on [GitHub](#)
-

Retrieved from "[https://en.wikipedia.org/w/index.php?title=React_\(software\)&oldid=1364591825](https://en.wikipedia.org/w/index.php?title=React_(software)&oldid=1364591825)"